Guia de Trabajo Semana 9 (Encapsulamiento)

1. Resultados de Aprendizaje Esperados (RAE)

Al finalizar el desarrollo completo de esta guía práctica y sus experiencias de laboratorio, el estudiante estará en capacidad de:

- **RAE 1 (Conceptual):** Comprender y diferenciar el principio de Ocultamiento de Información (Information Hiding) y la preservación de Invariantes de Clase, reconociendo el encapsulamiento como la definición de una interfaz pública de servicios coherente con las responsabilidades del objeto.
- **RAE 2 (Modelado):** Modelar en diagramas de clases UML Nivel 1 el encapsulamiento estricto, empleando visibilidad privada (-) para el estado, pública (+) para la interfaz de servicios y aplicando la convención de no redundancia gráfica y correspondencia exacta 1:1 con el código.
- **RAE 3 (Procedimental):** Implementar en Java clases robustas con atributos privados, validación defensiva en constructores (Fail-Fast mediante excepciones), métodos consultores sin efectos secundarios, métodos modificadores con retorno booleano, cálculo dinámico de atributos derivados y protección frente a fuga de colecciones.
- **RAE 4 (Metacognitivo):** Diseñar y ejecutar matrices de prueba con análisis de casos límite (valores no finitos, nulos y vacíos), verificando la invarianza del estado ante operaciones rechazadas y utilizando NotebookLM como tutor socrático de auditoría arquitectónica.

2. Actividad Diagnóstica: El Riesgo del Acceso No Controlado

En la comercialización de productos agroecológicos de la Universidad de la Amazonia (como la pulpa de arazá, copoazú o queso caqueteño), la caja de recaudo no se deja abierta y desatendida en el mostrador. Si cualquier transeúnte pudiera tomar dinero o ingresar billetes falsos sin control, la administración colapsaría de inmediato. En el diseño de software orientado a objetos, el estado interno de una entidad representa ese mismo activo crítico.

2.1 Análisis de Código Inseguro (Antipatrón de Diseño)

Considere el siguiente fragmento donde los atributos fueron declarados con visibilidad pública:

```
// ANTIPATRÓN: Atributos públicos expuestos sin control de acceso
public class ProductoInseguro {
    public String codigo;
    public String nombre;
    public double precioUnitario; // Atributo público: cualquier clase puede modificarlo
}

// Fragmento de ejecución en clase de prueba:
ProductoInseguro p = new ProductoInseguro();
p.codigo = "P001";
p.nombre = "";              // Nombre en blanco: no identifica ningún producto
p.precioUnitario = -15000.0;   // PRECIO NEGATIVO: El compilador NO marca error sintáctico
p.precioUnitario = Double.POSITIVE_INFINITY; // VALOR INFINITO: Provoca cálculos no finitos
```

Pregunta de Reflexión Inicial: ¿Por qué el compilador de Java acepta un precio de -15.000 COP o infinito? Porque sintácticamente cumplen con el tipo de dato primitivo double. El compilador verifica

tipos de datos, pero desconoce las reglas de negocio de la comercializadora. La responsabilidad de impedir estados corruptos recae en el diseño arquitectónico de la clase.

2.2 Ejercicio Diagnóstico Previo.

Antes de avanzar a la implementación guiada, responda en su cuaderno de trabajo las siguientes preguntas:

1. Identifique al menos tres problemas concretos que ocurrirían en la facturación si ProductoInseguro se conecta con las clases DetalleFactura y Factura.

2. Redacte en lenguaje natural las reglas de negocio que deberían cumplirse estrictamente para que un producto regional sea considerado válido.

3. Fundamentación Conceptual y Principios de Diseño

3.1 Precisión del Concepto de Encapsulamiento

El encapsulamiento no se reduce a declarar atributos como private y generar automáticamente métodos get y set. Concebir una clase de ese modo la convierte en una estructura pasiva de datos. El **encapsulamiento** es la integración de los datos y las operaciones que actúan sobre ellos dentro de un módulo cohesivo, exponiendo una **interfaz pública de servicios coherente** con las responsabilidades de la clase y manteniendo ocultos los detalles internos de implementación.

> **Principio de Ocultamiento de Información vs. Preservación de Invariantes**
> • Ocultamiento de Información (Information Hiding): Es la restricción técnica de acceso directo mediante el modificador private. Evita que clases externas conozcan cómo está estructurada internamente la memoria o dependan de representaciones concretas.
> • Preservación de Invariantes: Es la lógica de control implementada en constructores y métodos para garantizar que las reglas de consistencia del objeto jamás se violen durante su ciclo de vida.

3.2 Modificadores de Acceso y Distinción entre Visibilidad y Navegabilidad

Es fundamental distinguir dos conceptos que suelen confundirse en el modelado:

• **Visibilidad (+, -, #):** Aplica a los miembros internos de la clase (atributos y métodos). Define qué clases tienen autorización para acceder a un miembro específico.

• **Navegabilidad y Relaciones:** Aplica a las asociaciones estructurales entre clases. Una flecha indica qué clase conoce a otra para colaborar, independientemente de que los atributos internos de ambas permanezcan privados.

Modificador Java	Símbolo UML	Nivel de Acceso	Criterio de Uso Arquitectónico
private	- (menos)	Únicamente la propia clase	Estándar para todos los atributos de instancia y métodos auxiliares de validación interna.
public	+ (más)	Cualquier clase del sistema	Reservado para constructores y métodos que integran la interfaz de servicios.
protected	# (numeral)	Misma clase, paquete y subclases	Uso restringido a jerarquías de herencia (Unidad IV del curso).

3.3 Clasificación Funcional de Métodos en una Arquitectura Encapsulada

Para interactuar con un objeto cuyo estado está oculto, diseñamos métodos públicos clasificados en cuatro categorías estratégicas:

- **1. Constructor:** Método especial que se ejecuta automáticamente al instanciar un objeto con el operador new. Su misión es garantizar que el objeto nazca en un estado válido y completo, impidiendo referencias nulas o datos corruptos.
- **2. Métodos Consultores (Getters):** Permiten al exterior conocer el valor de un atributo sin darle la capacidad de alterarlo. Siguiendo el estándar de Jorge Villalobos, se nombran habitualmente como darNombreAtributo() o getNombreAtributo().
- **3. Métodos Modificadores Controlados (Setters):** Permiten solicitar un cambio en el estado del objeto. A diferencia de un simple asignador ciego, este método DEBE evaluar los invariantes de negocio mediante condicionales antes de aceptar la modificación, retornando frecuentemente un boolean que confirme si el cambio fue exitoso.
- **4. Métodos de Negocio y Atributos Derivados:** Aquellos datos que son el resultado de un cálculo matemático o lógico basado en otros atributos (por ejemplo, el subtotal en DetalleFactura o el total en Factura) NUNCA deben tener un método modificador (setSubtotal()). El valor debe calcularse en tiempo de ejecución para evitar inconsistencias.

Para mantener una política consistente de gestión de estados anómalos, la arquitectura adopta dos mecanismos según la etapa del ciclo de vida del objeto:

- **En el Constructor (Principio Fail-Fast mediante Excepciones):** Un objeto no debe nacer en memoria en un estado corrupto o incompleto. Si los parámetros de inicialización son nulos o violan las reglas de validez, el constructor debe lanzar IllegalArgumentException para abortar la instanciación y evitar que existan referencias inválidas en la memoria Heap.
- **En Métodos Modificadores (Preservación del Estado mediante Booleanos):** Cuando un objeto ya existe en un estado válido y recibe una solicitud de cambio con valores inadmisibles, el método rechaza la modificación retornando false. Esto deja intacto el estado previo y permite que la aplicación continúe su flujo sin interrupciones abruptas.

4. Modelado Arquitectónico en UML (Caso Tienda Amazónica)

El diseño arquitectónico del dominio comercial se especifica a continuación. Existe correspondencia exacta 1:1 entre los nombres, tipos y firmas del modelo y su implementación en código Java.

Convención de Modelado UML Adoptada en el Curso

• Regla de No Redundancia Gráfica: En este curso (siguiendo las directrices pedagógicas), si se traza una línea de asociación estructural entre dos clases, no se duplica esa relación escribiendo un atributo de tipo objeto en el compartimento de variables. La línea ya modela la referencia.

• Cardinalidad 1..* de Factura a DetalleFactura: Representa la regla de negocio para una factura válida y emitida comercialmente. En memoria, la colección se inicializa vacía al instanciar la Factura y se poblará progresivamente antes del cierre.

• Tipo de dato de fecha: En este laboratorio se emplea String para mantener el foco en el encapsulamiento básico. En aplicaciones profesionales de producción se recomienda java.time.LocalDate para garantizar consistencia temporal.

Clase del Dominio	Atributos Privados (-)	Métodos Públicos (+) y Responsabilidad
Producto	- codigo: String - nombre: String - precioUnitario: double	+ Producto(codigo: String, nombre: String, precio: double) + darCodigo(): String + darNombre(): String + darPrecioUnitario(): double + cambiarPrecioUnitario(nuevoPrecio: double): boolean
Cliente	- cedula: String - nombre: String - correo: String	+ Cliente(cedula: String, nombre: String, correo: String) + darCedula(): String + darNombre(): String + darCorreo(): String + cambiarCorreo(nuevoCorreo: String): boolean
DetalleFactura	- cantidad: int (Asociación 1 a 1 hacia Producto)	+ DetalleFactura(cantidad: int, producto: Producto) + darCantidad(): int + darProducto(): Producto + darSubtotal(): double (Atributo derivado) + modificarCantidad(nuevaCantidad: int): boolean
Factura	- numeroFactura: int - fecha: String (Asociación 1 a 1 hacia Cliente) (Colección 1..* hacia DetalleFactura)	+ Factura(numero: int, fecha: String, cliente: Cliente) + darNumeroFactura(): int + darFecha(): String + darCliente(): Cliente

+ darDetalles(): List<DetalleFactura>
+ agregarDetalle(detalle: DetalleFactura): boolean
+ darTotal(): double (Atributo derivado)

5. Matriz de Distribución de Responsabilidades

Para evitar la concentración indebida de lógica en una sola clase, las responsabilidades se distribuyen conforme a la siguiente matriz:

Clase	Responsabilidad de Validación	Responsabilidad de Cálculo / Custodia
Producto	Valida código no vacío, nombre no vacío y precio finito estrictamente positivo (> 0).	Provee el precio unitario base y autoriza ajustes controlados.
Cliente	Valida cédula no vacía, nombre no vacío y formato preliminar del correo electrónico.	Mantiene la identidad verificada del comprador.
DetalleFactura	Valida cantidad positiva (>= 1) y referencia no nula hacia un Producto existente.	Calcula dinámicamente el subtotal multiplicando cantidad por el precio vigente del producto.
Factura	Valida número positivo, fecha no vacía, cliente no nulo y detalles válidos.	Custodia la colección de detalles impidiendo accesos externos no autorizados y consolida el total de la venta.

6. Implementación en Java: De la Demostración Guiada a la Autonomía

6.1 Implementación de Referencia: Clase Producto.java

Observe en esta implementación de referencia cómo se integran Double.isFinite() para evitar números no finitos, el lanzamiento de excepciones en el constructor y el retorno booleano en el método mutador:

```
package co.edu.uniamazonia.tiendaamazonica.mundo;

/**
 * Modela un producto agroecológico regional.
 * Aplica encapsulamiento estricto y protección de invariantes.
 */
public class Producto {

    // 1. Atributos privados
    private String codigo;
    private String nombre;
    private double precioUnitario;

    // 2. Constructor: Validación defensiva (Fail-Fast)
    public Producto(String codigo, String nombre, double precioUnitario) {
```

```
        if (codigo == null || codigo.trim().isEmpty()) {
            throw new IllegalArgumentException("El código no puede ser nulo ni vacío.");
        }
        if (nombre == null || nombre.trim().isEmpty()) {
            throw new IllegalArgumentException("El nombre no puede ser nulo ni vacío.");
        }
        if (!Double.isFinite(precioUnitario) || precioUnitario <= 0.0) {
            throw new IllegalArgumentException("El precio debe ser un número finito mayor a 0.");
        }
        this.codigo = codigo.trim();
        this.nombre = nombre.trim();
        this.precioUnitario = precioUnitario;
    }

    // 3. Métodos consultores (Queries sin efectos secundarios)
    public String darCodigo() {
        return this.codigo;
    }

    public String darNombre() {
        return this.nombre;
    }

    public double darPrecioUnitario() {
        return this.precioUnitario;
    }

    // 4. Método modificador controlado (Command con preservación de estado)
    public boolean cambiarPrecioUnitario(double nuevoPrecio) {
        if (Double.isFinite(nuevoPrecio) && nuevoPrecio > 0.0) {
            this.precioUnitario = nuevoPrecio;
            return true; // Modificación autorizada
        }
        // Si la validación falla, el estado anterior permanece intacto
        return false; // Modificación rechazada
    }
}
```

6.2 Trabajo Autónomo: Criterios de Aceptación para DetalleFactura y Factura

Para las clases DetalleFactura y Factura, no se entrega el código terminado. Usted debe implementar las clases en NetBeans asegurando el cumplimiento de los siguientes Criterios de Aceptación (CA):

- **CA-01 (DetalleFactura - Atributo Derivado):** La clase debe contener como atributos privados cantidad y producto. No debe declarar subtotal como variable de instancia. El método public double darSubtotal() debe retornar cantidad * producto.darPrecioUnitario().
- **CA-02 (DetalleFactura - Modificación Controlada):** El método public boolean modificarCantidad(int nuevaCantidad) debe aceptar el cambio solo si nuevaCantidad >= 1. Si el valor es inválido, debe retornar false y mantener la cantidad previa.
- **CA-03 (DetalleFactura - Seguridad de Referencia):** El método public Producto darProducto() retorna la referencia al producto asociado. Esto es seguro porque la clase Producto protege internamente su propio estado.

- **CA-04 (Factura - Incorporación Segura):** El método public boolean agregarDetalle(DetalleFactura detalle) debe validar que detalle != null antes de agregarlo con .add(). Si es nulo, retorna false.
- **CA-05 (Factura - Total Dinámico):** El método public double darTotal() debe iterar sobre la lista de detalles acumulando el resultado de darSubtotal() de cada renglón.

7. Protocolo de Pruebas: Verificación de Invarianza y Casos Límite

Un programa que compila no es garantía de software correcto. En este laboratorio debe estructurar sus pruebas bajo el protocolo formal de cuatro elementos: **Estado Inicial, Operación Realizada, Resultado Esperado y Resultado Obtenido**. Verifique especialmente la conservación del estado ante operaciones rechazadas:

Caso	Estado Inicial	Operación Realizada	Resultado Esperado	Resultado Obtenido
CP-01	Sin objeto instanciado.	new Producto("", "Arazá", 8500.0)	Lanzamiento de IllegalArgumentException. Objeto no creado.	Excepción confirmada. Mensaje descriptivo en consola.
CP-02	Sin objeto instanciado.	new Producto("P001", "Copoazú", -5000.0)	Lanzamiento de IllegalArgumentException por precio negativo.	Excepción confirmada. Instanciación prevenida.
CP-03	Producto P001 con precio = $8,500.0.	p.cambiarPrecioUnitario(-3000.0)	Retorna false. El precio del producto permanece exactamente en $8,500.0.	Retorna false. Invariante conservado ($8,500.0).
CP-04	Producto P001 con precio = $8,500.0.	p.cambiarPrecioUnitario(Double.NaN)	Retorna false. El precio permanece inalterado.	Retorna false. Invariante conservado.
CP-05	DetalleFactura con 2 unidades de P001.	det.modificarCantidad(0)	Retorna false. La cantidad permanece en 2 y subtotal en $17,000.0.	Retorna false. Estado y subtotal preservados.

8. Retos Autónomos de Programación

Desarrolle los siguientes retos en su proyecto de NetBeans:

Reto 1: Descuentos Promocionales en Producto (Invariantes y Rango)

• Requerimiento: Agregue a Producto el atributo privado descuentoPromocional (tipo double). Representa el porcentaje de descuento vigente para productos regionales (por ejemplo, 0.15 para el 15%).
• Regla de Negocio: El rango permitido es estrictamente entre 0.0 (0%) y 0.50 (50% de descuento máximo).
• Método Modificador: public boolean aplicarDescuento(double porcentaje). Si el porcentaje está fuera de [0.0, 0.50] o no es finito, retorna false y preserva el descuento anterior.
• Método Consultor: public double darPrecioConDescuento(). Calcula bajo demanda: precioUnitario * (1.0 - descuentoPromocional). No almacene el precio final como variable fija.

Reto 2: Validación Preliminar de Correo en Cliente

• Requerimiento: En la clase Cliente, implemente el método public boolean cambiarCorreo(String nuevoCorreo).
• Alcance Didáctico: Para el nivel de este ejercicio, se aplica una validación sintáctica simplificada: comprobar que la cadena no sea nula, contenga al menos un carácter '@' y al menos un punto '.' posterior al '@'.

Reto 3: Integridad de Factura y Prevención de Duplicados

• Regla Obligatoria: En Factura, el método agregarDetalle(DetalleFactura detalle) debe rechazar referencias nulas y garantizar que darDetalles() no exponga la colección interna.
• Mejora Avanzada: Implemente el método public boolean contieneProducto(String codigoProducto) para verificar si un producto ya está registrado en la factura sin exponer la lista.
• Política ante Duplicados: Diseñe el método para que, si el producto ya existe en la factura, no cree un nuevo renglón, sino que actualice la cantidad del detalle existente sumando las nuevas unidades.